

学号	
----	--

武汉理工大学

《程序综合设计实验》报告

学 院	计算机与人工智能学院
--------	------------

专 业	计算机类
--------	------

班 级	
--------	--

姓 名	
--------	--

指导教师	罗芳、郭小兵
------	--------

1 实验目的

1.1 实验背景

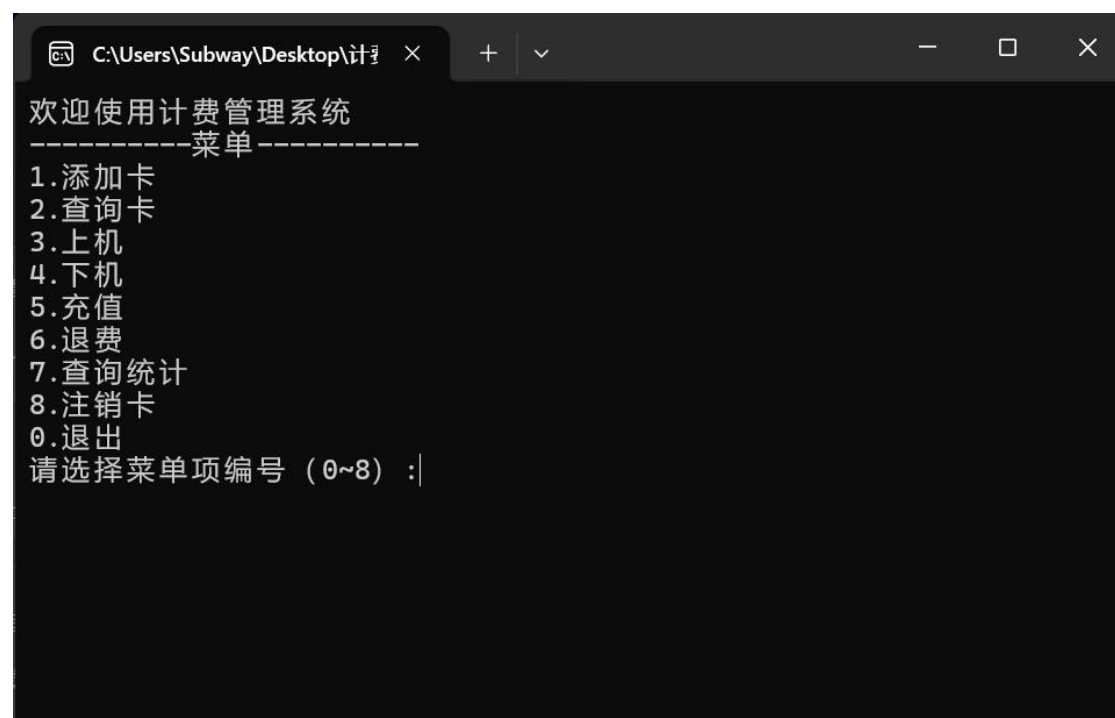
计费管理系统主要面对网吧、体育馆等根据时间来计费的场所。由于其在计费管理上所体现的突出的优越性，使得这类场所管理人员能够轻松管理，以达到高效、精准、自动化运营管理，大幅减少人工核算误差与人力成本，同时实现对营收数据、会员信息的统一管控，提升整体经营效率与服务质量。

1.2 实验目的

本次实验，通过开发计费管理系统软件，达到以下五个目的：

- 1) 理解 C 语言迭代开发的思想，逐步迭代完善系统功能；
- 2) 掌握需求分析、系统设计、编码实现、功能调试的完整 C 语言项目开发流程；
- 3) 熟练理解和掌握 C 语言的高级知识，如字符串、结构体、文件读写、动态内存管理、链表等技术；
- 4) 结合网吧等时间计费场景，实现核心计费与数据管理功能，提升 C 语言解决实际业务问题的应用能力。

2 系统功能与描述



```
C:\Users\Subway\Desktop\计费管理系统 >
欢迎使用计费管理系统
-----菜单-----
1.添加卡
2.查询卡
3.上机
4.下机
5.充值
6.退费
7.查询统计
8.注销卡
0.退出
请选择菜单项编号 (0~8) :|
```

上面是计费管理系统的主界面，下面对涉及到的 8 个功能及拓展功能进行简要的描述：

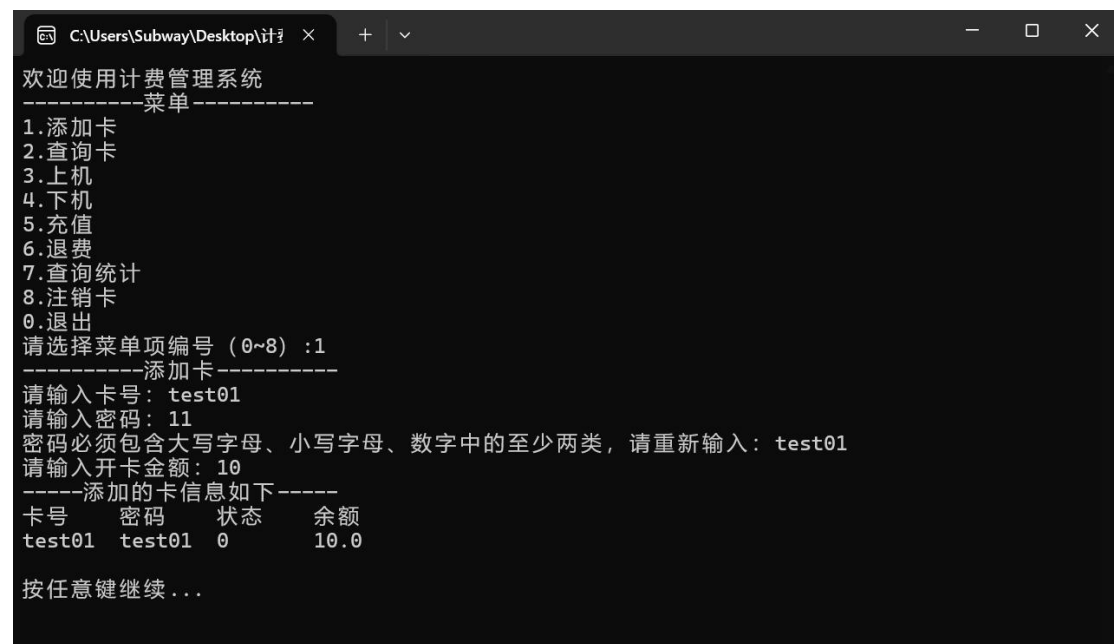
2.1 添加卡

用户在网吧上网需要卡，卡片信息包括卡号，密码，余额，状态等。在添加卡功能中，用户通过输入卡号，密码，开卡金额的信息，可完成开卡操作。

2.1.1 添加卡失败：

有多种情况会导致开卡失败：

1)

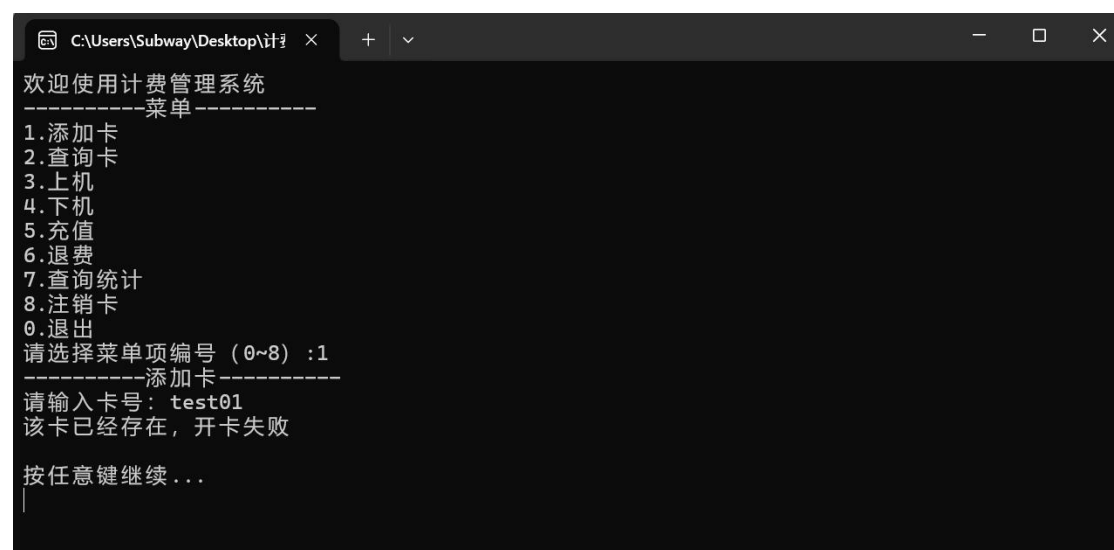


```
C:\Users\Subway\Desktop\计费管理>
欢迎使用计费管理系统
-----菜单-----
1.添加卡
2.查询卡
3.上机
4.下机
5.充值
6.退费
7.查询统计
8.注销卡
0.退出
请选择菜单项编号 (0~8) :1
-----添加卡-----
请输入卡号: test01
请输入密码: 11
密码必须包含大写字母、小写字母、数字中的至少两类, 请重新输入: test01
请输入开卡金额: 10
-----添加的卡信息如下-----
卡号    密码    状态    余额
test01  test01  0       10.0

按任意键继续...
```

当密码未包含大写字母、小写字母、数字中的至少两类时，会要求用户重新输入符合安全规范的密码

2)

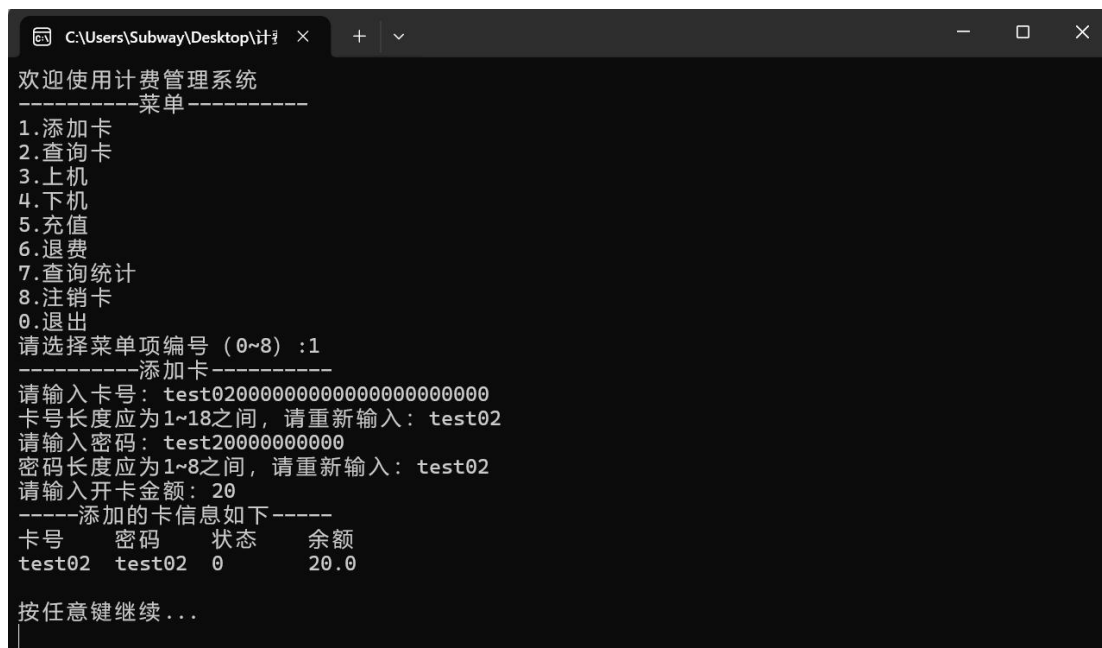


```
C:\Users\Subway\Desktop\计费管理>
欢迎使用计费管理系统
-----菜单-----
1.添加卡
2.查询卡
3.上机
4.下机
5.充值
6.退费
7.查询统计
8.注销卡
0.退出
请选择菜单项编号 (0~8) :1
-----添加卡-----
请输入卡号: test01
该卡已经存在, 开卡失败

按任意键继续...
|
```

当输入的卡号已经存在时，开卡失败

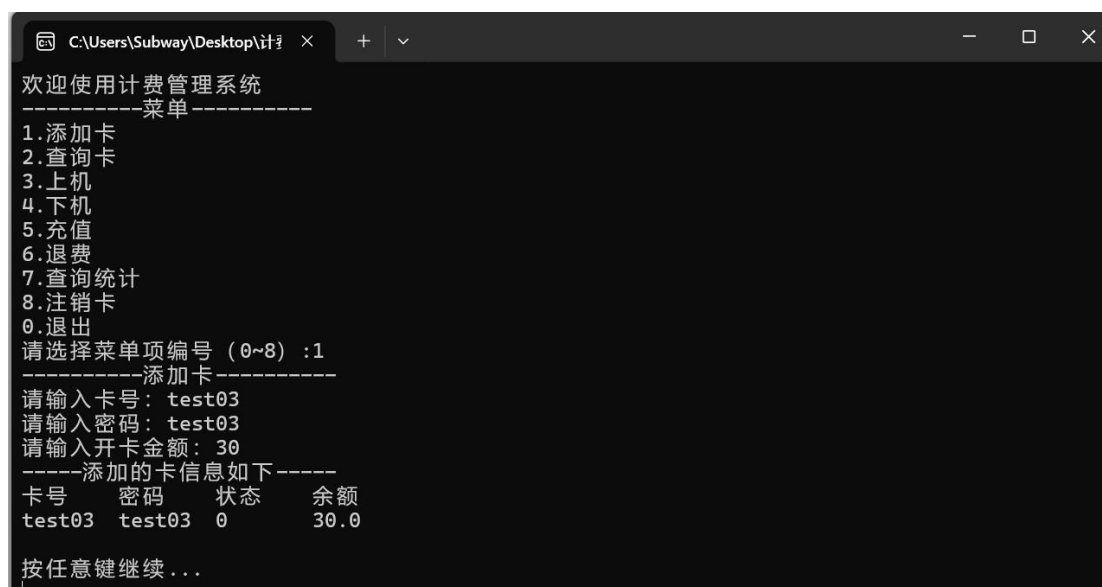
3)



```
C:\Users\Subway\Desktop\计费管理>
欢迎使用计费管理系统
-----菜单-----
1.添加卡
2.查询卡
3.上机
4.下机
5.充值
6.退费
7.查询统计
8.注销卡
0.退出
请选择菜单项编号 (0~8) :1
-----添加卡-----
请输入卡号: test02000000000000000000000000000000
卡号长度应为1~18之间, 请重新输入: test02
请输入密码: test20000000000000000000000000000000
密码长度应为1~8之间, 请重新输入: test02
请输入开卡金额: 20
-----添加的卡信息如下-----
卡号    密码    状态    余额
test02  test02  0       20.0
按任意键继续...
```

当输入的卡号超过 18 位或者密码超过 8 位时，不符合规范，需要重新输入符合长度规范的卡号和密码

2.1.2 添加卡成功:



```
C:\Users\Subway\Desktop\计费管理>
欢迎使用计费管理系统
-----菜单-----
1.添加卡
2.查询卡
3.上机
4.下机
5.充值
6.退费
7.查询统计
8.注销卡
0.退出
请选择菜单项编号 (0~8) :1
-----添加卡-----
请输入卡号: test03
请输入密码: test03
请输入开卡金额: 30
-----添加的卡信息如下-----
卡号    密码    状态    余额
test03  test03  0       30.0
按任意键继续...
```

当输入的卡号、密码都符合规范且卡号未被使用时，开卡成功。

2.2 查询卡

这里采取了模糊查询的策略。用户输入卡号的一部分，通过子串匹配和两遍遍历算法匹配系统中存有的卡号，找到所有符合要求的卡片，并显示信息。

```
C:\Users\Subway\Desktop\计费管理>
欢迎使用计费管理系统
-----菜单-----
1. 添加卡
2. 查询卡
3. 上机
4. 下机
5. 充值
6. 退费
7. 查询统计
8. 注销卡
0. 退出
请选择菜单项编号 (0~8) :2
-----查询卡-----
请输入要查询的卡号: test
-----查询到的卡信息如下-----
卡号  密码  卡状态  余额  累计使用  使用次数  上次使用时间
test01 test01 0      10.0  0.00      0          2026-04-18 12:46
卡号  密码  卡状态  余额  累计使用  使用次数  上次使用时间
test02 test02 1      20.0  0.00      1          2026-04-18 15:04
卡号  密码  卡状态  余额  累计使用  使用次数  上次使用时间
test03 test03 0      30.0  0.00      0          2026-04-18 12:58
按任意键继续 ...
```

2.3 上机

当用户上网时，进行上机操作

2.3.1 上机失败

上机失败有多种可能：

1)

```
C:\Users\Subway\Desktop\计费管理>
欢迎使用计费管理系统
-----菜单-----
1. 添加卡
2. 查询卡
3. 上机
4. 下机
5. 充值
6. 退费
7. 查询统计
8. 注销卡
0. 退出
请选择菜单项编号 (0~8) :3
请输入卡号: test1
请输入密码: 12
卡号错误!
```

链表中没有该卡号

2)

```
C:\Users\Subway\Desktop\计费管理>
欢迎使用计费管理系统
-----菜单-----
1. 添加卡
2. 查询卡
3. 上机
4. 下机
5. 充值
6. 退费
7. 查询统计
8. 注销卡
0. 退出
请选择菜单项编号 (0~8) :3
请输入卡号: test02
请输入密码: 123
密码错误!

按任意键继续...
```

有该卡号，但是密码错误

3)

该卡已上机

```
C:\Users\Subway\Desktop\计费管理>
欢迎使用计费管理系统
-----菜单-----
1. 添加卡
2. 查询卡
3. 上机
4. 下机
5. 充值
6. 退费
7. 查询统计
8. 注销卡
0. 退出
请选择菜单项编号 (0~8) :3
请输入卡号: test01
请输入密码: test01
该卡已上机!
```

4)

```
C:\Users\Subway\Desktop\计费管理 >
欢迎使用计费管理系统
-----菜单-----
1.添加卡
2.查询卡
3.上机
4.下机
5.充值
6.退费
7.查询统计
8.注销卡
0.退出
请选择菜单项编号 (0~8) :3
请输入卡号: test04
请输入密码: test04
该卡无法使用!
按任意键继续...
```

卡被注销或失效

5)

```
C:\Users\Subway\Desktop\计费管理 >
欢迎使用计费管理系统
-----菜单-----
1.添加卡
2.查询卡
3.上机
4.下机
5.充值
6.退费
7.查询统计
8.注销卡
0.退出
请选择菜单项编号 (0~8) :3
请输入卡号: test05
请输入密码: test05
余额不足, 无法上机!
按任意键继续...
```

卡余额不足

2.3.2 上机成功:

```
C:\Users\Subway\Desktop\计费管理 >
欢迎使用计费管理系统
-----菜单-----
1.添加卡
2.查询卡
3.上机
4.下机
5.充值
6.退费
7.查询统计
8.注销卡
0.退出
请选择菜单项编号 (0~8) :3
请输入卡号: test01
请输入密码: test01
上机成功
-----上机信息如下-----
卡号      状态      余额
test01    1          9.0
```

2.4 下机

下机功能的实现与上机相似，也会出现下机失败的情况（如密码错误、卡号错误、正在上机等情况），当下机成功，停止计费，生成账单并扣费

```
C:\Users\Subway\Desktop\计费管理 >
欢迎使用计费管理系统
-----菜单-----
1.添加卡
2.查询卡
3.上机
4.下机
5.充值
6.退费
7.查询统计
8.注销卡
0.退出
请选择菜单项编号（0~8）：4
-----下机-----
请输入卡号：test02
请输入密码：test02
下机成功
-----下机信息如下-----
卡号    状态    余额    应付金额    开始时间    结束时间
test02  0       18.5    1.5         2026-04-18 15:04    2026-04-18 15:37
按任意键继续...
```

当余额不足以支付在这次费用，会显示

```
C:\Users\Subway\Desktop\计费管理 >
欢迎使用计费管理系统
-----菜单-----
1.添加卡
2.查询卡
3.上机
4.下机
5.充值
6.退费
7.查询统计
8.注销卡
0.退出
请选择菜单项编号（0~8）：4
-----下机-----
请输入卡号：test06
请输入密码：test06
余额不足，无法下机！
按任意键继续...
```


2.5 充值

用户输入金额，向卡片中添加余额。

```
C:\Users\Subway\Desktop\计费管理 >
按任意键继续...
欢迎使用计费管理系统
-----菜单-----
1. 添加卡
2. 查询卡
3. 上机
4. 下机
5. 充值
6. 退费
7. 查询统计
8. 注销卡
0. 退出
请选择菜单项编号 (0~8) :5
-----充值-----
请输入卡号: test01
请输入密码: test01
请输入充值金额: 10
充值成功
-----充值信息如下-----
卡号      充值金额      余额      时间
test01    10.0            19.0      2026-04-18 16:25
按任意键继续...
```

2.6 退费

用户输入金额，取出卡片中的余额。

```
C:\Users\Subway\Desktop\计费管理 >
欢迎使用计费管理系统
-----菜单-----
1. 添加卡
2. 查询卡
3. 上机
4. 下机
5. 充值
6. 退费
7. 查询统计
8. 注销卡
0. 退出
请选择菜单项编号 (0~8) :6
-----退费-----
请输入卡号: test03
请输入密码: test03
退费成功
-----退费信息如下-----
卡号      退费金额      余额      时间
test03    30.0            0.0      2026-04-18 16:11
按任意键继续...
```

2.7 查询统计

2.7.1 查询某卡片某时间段内的消费记录

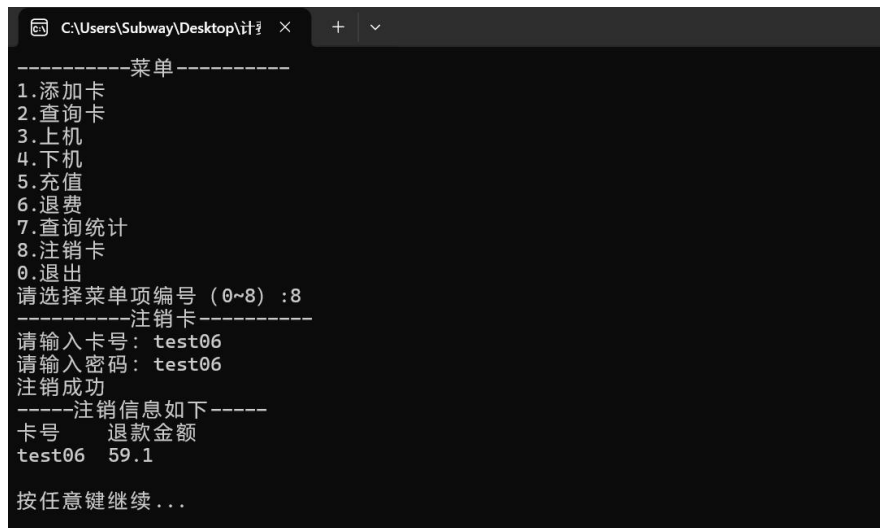
```
C:\Users\Subway\Desktop\计费管理
欢迎使用计费管理系统
-----菜单-----
1.添加卡
2.查询卡
3.上机
4.下机
5.充值
6.退费
7.查询统计
8.注销卡
0.退出
请选择菜单项编号 (0~8) :7
-----查询统计-----
1.查询卡消费情况
2.查询营业额
0.返回
请选择: 1
-----查询卡消费情况-----
请输入卡号: test01
请输入密码: test01
请输入开始时间 (年-月-日): 2026-01-01
请输入结束时间 (年-月-日): 2026-05-01
查询成功
-----卡消费情况如下-----
卡号      开始时间      结束时间      总消费额
test01    2026-01-01      2026-05-01      1.0
按任意键继续...
```

2.7.1 查询商家某时间段内的营业额

```
C:\Users\Subway\Desktop\计费管理
欢迎使用计费管理系统
-----菜单-----
1.添加卡
2.查询卡
3.上机
4.下机
5.充值
6.退费
7.查询统计
8.注销卡
0.退出
请选择菜单项编号 (0~8) :7
-----查询统计-----
1.查询卡消费情况
2.查询营业额
0.返回
请选择: 2
-----查询营业额-----
请输入开始时间 (年-月-日): 2026-01-01
请输入结束时间 (年-月-日): 2026-05-01
-----营业额情况如下-----
开始时间      结束时间      营业额
2026-01-01      2026-05-01      2.5
按任意键继续...
```

2.8 注销卡

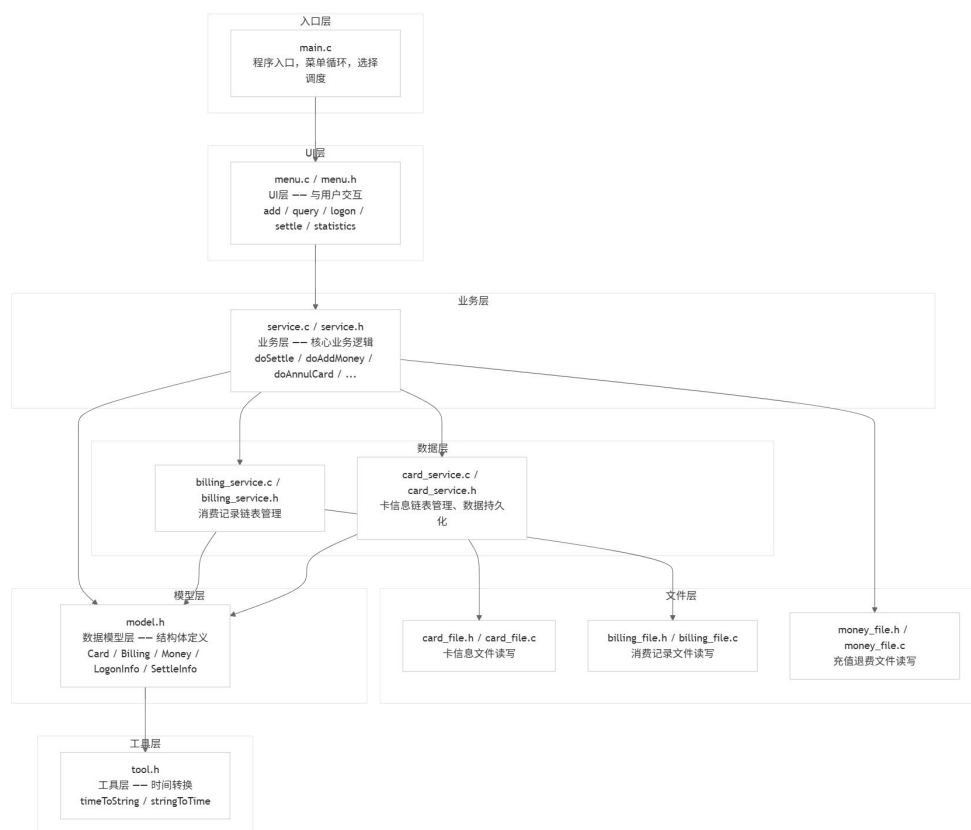
设置卡片状态为注销，此后无法进行上机下机等等操作，同时退款卡内的余额。



```
C:\Users\Subway\Desktop\计... × + v
-----菜单-----
1.添加卡
2.查询卡
3.上机
4.下机
5.充值
6.退费
7.查询统计
8.注销卡
0.退出
请选择菜单项编号 (0~8) :8
-----注销卡-----
请输入卡号: test06
请输入密码: test06
注销成功
-----注销信息如下-----
卡号      退款金额
test06    59.1
按任意键继续...
```

3 设计与实现

3.1 文件结构设计



3.2 数据结构设计

3.2.1 卡结构体:

```
typedef struct Card
{
    char aName[10];    //卡号
    char aPwd[8];      //密码
    int nStatus;        //卡状态 (0-未上机; 1-正在上机; 2-已注销; 3-失效)
    time_t tStart;      //开卡时间
    time_t tEnd;        //卡的截止时间
    float fTotalUse;    //累计金额
    time_t tLast;      //最后使用时间
}
```

```

    int nUseCount;    //使用次数
    float fBalance;   //余额
    int nDel;         //删除标志 (0-未删除; 1-已删除)
}Card;

```

3.2.2 账单结构体:

```

typedef struct Billing {
    char aName[18];    //卡号
    time_t tStart;     //上机时间
    time_t tEnd;       //下机时间
    float fAmount;     //本次使用金额
    int nStatus;       //消费状态 (0-未结算; 1-已结算)
    int nDel;          //删除标志 (0-未删除; 1-已删除)
}Billing;

```

3.2.3 上/下机结构体:

```

//上机信息结构体
typedef struct LogonInfo {
    char aName[18];    //卡号
    time_t tLogon;     //上机时间
    float fBalance;    //余额
}LogonInfo;

```

```

//下机信息结构体
typedef struct BillingNode {
    Billing data;
    struct BillingNode* next;
} BillingNode;

```

3.2.4 退费结构体:

```

//充值退费结构体
typedef struct Money {
    char aName[18];    //卡号
    float fMoney;      //充值或退费金额
    time_t tTime;      //充值或退费时间
    int nStatus;       //类型 (0-充值; 1-退费)
    int nDel;          //删除标志 (0-未删除; 1-已删除)
}Money;

```

```
//充值退费信息结构体
typedef struct MoneyInfo {
    char aName[18];    //卡号
    float fMoney;      //充值或退费金额
    float fBalance;    //余额
    time_t tTime;      //时间
}MoneyInfo;
```

3.2.5 注销卡结构体:

```
typedef struct AnnulInfo {
    char aName[18];    //卡号
    float fBalance;    //退款金额
}AnnulInfo;
```

3.2.6 消费信息结构体（用于查询统计）:

```
typedef struct ConsumptionInfo {
    char aName[18];    //卡号
    char startDate[20]; //开始时间
    char endDate[20];  //结束时间
    float fTotalConsumption; //总消费额
}ConsumptionInfo;
```

3.3 核心算法设计:

3.3.1 链表初始化与节点插入

1) 带头结点的链表初始化

```
int initCardList() {
    lpCardNode head = NULL;
    head = (lpCardNode)malloc(sizeof(CardNode));
    if (head != NULL) {
        head->next = NULL;
        cardList = head;
        return 1;
    }
    return 0;
}
```

链表采用带头结点的单向链表结构。cardList 为头指针，不存储实际数据，仅作为链表的入口。初始化时为头结点分配内存，并将其 next 指针置为 NULL。

这样做的好处是在插入和删除操作时不需要单独处理空链表的情况，代码逻辑更加统一。

2) 尾插法插入节点

```
int addCard(Card card) {
    if (saveCard(&card, CARDPATH) == 1) {
        lpCardNode newNode = (lpCardNode)malloc(sizeof(CardNode));
        if (newNode == NULL) return 1;
        memset(newNode, 0, sizeof(CardNode));
        newNode->data = card;
        newNode->next = NULL;

        // 找到链表尾部并插入
        lpCardNode tail = cardList;
        while (tail->next != NULL) tail = tail->next;
        tail->next = newNode;
        return 1;
    }
    return 0;
}
```

从头结点出发，通过 `tail->next != NULL` 找到当前链表的最后一个有效节点，然后将新节点接在其后面

3.3.2 文件批量读取与链表构建

1) 二进制文件大小计算与记录数推导

```
int readCard(Card* pCard, const char* pPath) {
    FILE* fp = NULL;
    fopen_s(&fp, pPath, "rb");
    if (fp == NULL) {
        return 0;
    }

    // 计算记录数
    fseek(fp, 0, SEEK_END);
    long nSize = ftell(fp);
    if (nSize <= 0) {
        fclose(fp);
        return 0;
    }
    long nCount = nSize / sizeof(Card);
    fseek(fp, 0, SEEK_SET);

    // 读取所有记录
    size_t nRead = fread(pCard, sizeof(Card), (size_t)nCount, fp);
}
```

```

fclose(fp);
if (nRead != (size_t)nCount) {
    return 0;
}
return 1;
}

```

通过 `fseek` 将文件指针移到末尾，用 `ftell` 获取文件总字节数。由于二进制文件中每条记录大小固定为 `sizeof(Card)`，因此记录数 = 文件总大小 / 单条记录大小。这是一种直接定长记录文件的读取算法，无需在文件中存储元数据（如记录数），仅依靠文件大小即可推算。

`getCardCount` 和 `getBillingCount` 也使用了相同的原理——通过 `ftell` 获取文件大小后除以单条记录大小。

3) 顺序表的批量加载与链表的批量构建

```

int getCard() {
    Card* pCard = NULL;
    lpCardNode node = NULL;
    lpCardNode cur = NULL;

    // 先释放已有链表
    if (cardList != NULL) {
        releaseCardList();
    }
    initCardList();

    // 读取文件中的卡数量
    int nCount = getCardCount(CARDPATH);
    if (nCount <= 0) return 0;

    // 分配内存并读取文件
    pCard = (Card*)malloc(nCount * sizeof(Card));
    if (pCard == NULL) {
        return 0;
    }
    if (readCard(pCard, CARDPATH) == 0) {
        free(pCard);
        return 0;
    }

    // 构建链表
    node = cardList;
    for (int i = 0; i < nCount; i++) {
        cur = (lpCardNode)malloc(sizeof(CardNode));
        if (cur == NULL) {
            releaseCardList();

```



```

        free(pCard);
        return 0;
    }
    memset(cur, 0, sizeof(CardNode));
    cur->data = pCard[i];
    cur->next = NULL;
    node->next = cur;
    node = cur;
}
free(pCard);
pCard = NULL;
return 1;
}

```

采用顺序表 → 链表” 两次遍历策略：

第一次遍历：调用 `getCardCount` 获取文件中的记录数，分配相应大小的数组，一次性从文件读入内存（顺序表）。。

第二次遍历：遍历顺序表，将每条记录复制到新分配的链表节点中，构建链表。

3.3.3 链表节点的线性查找

1) 精确查询

```

Card* queryCard(const char* aName) {
    if (aName == NULL || cardList == NULL) {
        return NULL;
    }
    lpCardNode cur = cardList->next;
    while (cur != NULL) {
        if (strcmp(cur->data.aName, aName) == 0 && cur->data.nDel == 0) {
            return &cur->data;
        }
        cur = cur->next;
    }
    return NULL;
}

```

顺序遍历链表，逐个与目标关键字（卡号）进行比较。通过 `nDel == 0` 过滤已注销的记录。

2) 模糊查询（两遍遍历 + 字符串包含匹配）

```

Card* queryCards(const char* pName, int* pIndex) {
    if (pIndex) *pIndex = 0;
    if (pName == NULL || cardList == NULL) {
        return NULL;
    }

    // 第一遍：统计匹配数量

```

```

lpCardNode cur = cardList->next;
int count = 0;
while (cur != NULL) {
    if (strstr(cur->data.aName, pName) != NULL && cur->data.nDel == 0) {
        count++;
    }
    cur = cur->next;
}

if (count == 0) {
    return NULL;
}

// 分配内存
Card* pCard = (Card*)malloc(count * sizeof(Card));
if (pCard == NULL) return NULL;

// 第二遍：复制匹配的数据
int i = 0;
cur = cardList->next;
while (cur != NULL) {
    if (strstr(cur->data.aName, pName) != NULL && cur->data.nDel == 0) {
        pCard[i++] = cur->data;
    }
    cur = cur->next;
}

if (pIndex) *pIndex = count;
return pCard;
}

```

模糊查询使用 strstr 函数判断子串是否包含在主串中，采用两遍遍历策略：

第一遍：遍历链表统计匹配数量，以确定需要分配的内存大小。

第二遍：再次遍历链表，将匹配的数据复制到动态分配的数组中返回。

两遍遍历的目的：避免事先无法确定数组大小的问题。

3.3.4 多条件状态机验证

```

// 查找卡并验证
cardNode = cardList->next;
while (cardNode != NULL) {
    // 检查密码是否正确
    if (strcmp(cardNode->data.aName, aName) == 0 && strcmp(cardNode->data.aPwd,
aPwd) != 0) {
        return WRONGPWD;
    }
}

```

```

    }
    // 检查卡是否存在且可用的各项条件
    if (strcmp(cardNode->data.aName, aName) == 0 && strcmp(cardNode->data.aPwd,
aPwd) == 0) {
        if (cardNode->data.nStatus == 2 || cardNode->data.nStatus == 3 ||
cardNode->data.nDel == 1) {
            return INVALID;
        }
        if (cardNode->data.nStatus == 1) {
            return ONSTATUS;
        }
        if (cardNode->data.fBalance <= 0) {
            return ENOUGHMONEY;
        }
        break;
    }
    cardNode = cardNode->next;
    nIndex++;
}

```

在 `doLogon`（上机）函数中，使用多层 `if-else` 状态机进行身份和状态的合法性验证。验证顺序为：

密码错误（卡号存在但密码不匹配）→ `WRONGPWD`

卡无效（已注销/已删除）→ `INVALID`

卡已上机（重复上机）→ `ONSTATUS`

余额不足（无法上机）→ `ENOUGHMONEY`

这种顺序设计确保了用户体验友好——先检查最容易纠正的错误（密码输错），再检查无法操作的错误状态。

同样的多条件验证模式也出现在 `doSettle`、`doAddMoney`、`doRefundMoney`、`doAnnulCard` 中，区别仅在于允许的状态值不同。

4.开发难点与体会

4.1 二进制记录文件的读取与存储

将数据结构体以二进制形式直接写入文件，再从文件读出重建链表，是本系统的一个难点。

写入文件比较直接，使用 `fwrite` 按结构体大小整块写入即可：

```

int saveCard(const Card* pCard, const char* pPath) {
    FILE* fp = NULL;
    fopen_s(&fp, pPath, "ab");
    if (fp == NULL) {

```

```

        return 0;
    }
    fwrite(pCard, sizeof(Card), 1, fp);
    fclose(fp);
    return 1;
}

```

读取的难点在于：二进制文件不像文本文件那样可以逐行读取。每条 Card 记录的大小固定为 sizeof(Card)，因此需要先通过 ftell 获取文件总字节数，再除以单条记录大小来推导记录数：

```

int readCard(Card* pCard, const char* pPath) {
    FILE* fp = NULL;
    fopen_s(&fp, pPath, "rb");
    if (fp == NULL) {
        return 0;
    }

    // 计算记录数
    fseek(fp, 0, SEEK_END);
    long nSize = ftell(fp);
    if (nSize <= 0) {
        fclose(fp);
        return 0;
    }
    long nCount = nSize / sizeof(Card);
    fseek(fp, 0, SEEK_SET);

    // 读取所有记录
    size_t nRead = fread(pCard, sizeof(Card), (size_t)nCount, fp);
    fclose(fp);
    if (nRead != (size_t)nCount) {
        return 0;
    }
    return 1;
}

```

更新文件同样利用记录定长的特点，通过索引号直接定位：

```

int updateCard(const Card* pCard, const char* pPath, int nIndex) {
    FILE* fp = NULL;
    fopen_s(&fp, pPath, "rb+");
    if (fp == NULL) {
        return 0;
    }
    long lPosition = nIndex * sizeof(Card);
    fseek(fp, lPosition, SEEK_SET);
    fwrite(pCard, sizeof(Card), 1, fp);
}

```

```

    fclose(fp);
    return 1;
}

```

与文本文件逐行解析的方式不同，程序采用二进制直接定长记录文件存储，核心优势是无需考虑字符串解析的边界问题（如空格、## 分隔符的处理），但需要精确维护记录数与索引的对应关系，否则错误的索引会导致数据覆盖到错误的记录上。

4.2 时间数据的二进制存储

时间数据在本系统中以 `time_t` 类型直接作为结构体成员写入二进制文件，`fread/fwrite` 会完整保留其二进制表示，无需字符串转换。这与参考代码中字符串形式的 "年-月-日 时:分" 需要单独解析的逻辑不同。

但是在界面展示时，仍然需要将 `time_t` 转换为可读字符串。在 `doQueryCardConsumption` 和 `doQueryTurnover` 中，用户输入的是日期字符串（如 "2026-01-01"），需要转换为 `time_t` 才能参与比较：

```

    struct tm startTm = { 0 }, endTm = { 0 };
    sscanf(pStartDate, "%d-%d-%d", &startTm.tm_year, &startTm.tm_mon,
&startTm.tm_mday);
    sscanf(pEndDate, "%d-%d-%d", &endTm.tm_year, &endTm.tm_mon, &endTm.tm_mday);
    startTm.tm_year -= 1900;
    endTm.tm_year -= 1900;
    startTm.tm_mon -= 1;
    endTm.tm_mon -= 1;
    startTm.tm_hour = 0;
    startTm.tm_min = 0;
    startTm.tm_sec = 0;
    endTm.tm_hour = 23;
    endTm.tm_min = 59;
    endTm.tm_sec = 59;
    time_t startTime = mktime(&startTm);
    time_t endTime = mktime(&endTm);

```

排查时间解析错误时，可以先用 `printf` 打印 `startTm` 的各个字段值，确认年月日是否正确解析。特别要注意两个系统的偏移：`tm_year` 需要减去 1900，`tm_mon` 需要减去 1，漏掉这两个操作会导致解析出的时间偏差数十年甚至数年。

5 实验总结

这次的实验，对于一个未接触过数据结构的大一学生是很有挑战性的。首先遇到的难点就是链表，虽然对于链表的定义和基本操作，在课堂上已经有所接触，但是真正运用到实际的系统开发中，仍然感到力不从心。通过这次实验，我对于链表的创建、插入、查找、释放等基本操作有了更加深刻的理解，也体会到了指针操作在 C 语言中的核心地位。

在最初的实现中，我对链表节点插入时 `next` 指针的初始化不够重视，导致程序在运行过程中频繁出现死循环。经过反复调试，我逐渐认识到：每创建一个新节点，必须将其 `next` 指针显式置为 `NULL`，这不仅是链表遍历退出的必要条件，更是避免野指针和程序崩溃的关键。这一细节看似微不足道，却花费了我大量的时间去排查和理解。

再者，这次实验使我更加熟悉了文件的读取和写入。文件的操作在实际的项目编写中很重要，但这是过往理论课学习容易被忽视的部分。如果不能正确的从文件中读取和写入数据，会直接影响整个项目的功能实现。在本次实验中，程序采用二进制直接定长记录的方式对数据进行持久化存储。通过 `fseek` 和 `ftell` 的配合，我学会了如何通过文件大小推导记录数量，如何通过索引号直接定位文件中的一条记录。通过完成整个计费管理系统的开发，我对模块化设计有了初步的体会。不同于 `online judgement` 平台上刷题，程序设计涉及从数据模型 (`model.h`) 到文件读写 (`card_file.c`、`billing_file.c`) 再到业务逻辑 (`service.c`)，每一层的职责划分使得代码结构清晰，使得系统便于维护和管理。这次实验让我认识到，一个完整的系统不仅仅是功能的堆砌，更需要合理的数据结构设计、清晰的文件存储方案以及严谨的错误处理策略。

最后，感谢我的指导老师，在课堂上对我们的悉心指导和教诲，也感谢丰富的网络资源和大模型在本次实验代码调试过程中，提供了宝贵的思路与参考，帮助我更系统地梳理了程序背后的算法逻辑。

这次实验，是我第一次综合地开发一个系统，当然这个系统有很多不完善之处，也涵盖了我暂时难以完全理解的知识。在未来的学习中，我想，我会不断地学习新的知识，让自己在程序设计的道路上走得更远。

《程序综合设计实验》成绩评定表

班级：

姓名：

学号：

序号	评分项目	满分	实得分
1	需求分析合理	20	
2	算法描述清晰	30	
3	测试用例详尽	20	
4	实验总结完整	15	
5	报告格式规范	15	
		总得分	

验收记录：

指导教师签名：

2026 年 月 日